Course Title: Artificial Intelligence (2025)

Assignment 2: Informed Search, CSP, Adversarial Search

Theoretical Questions

Q1: Class Scheduling

Assume you are responsible for scheduling computer science classes that are held on Saturdays, Sundays, and Mondays. Five classes will be held on these days, and three professors will teach these classes. You are constrained by the fact that each professor can teach only one class at any given time, and the following constraints also apply:

The classes are:

1. Class 1 - Computer Fundamentals from 8 AM to 9 AM
2. Class 2 - Artificial Intelligence from 8:30 AM to 9:30 AM
3. Class 3 - Natural Language Processing from 9 AM to 10 AM
4. Class 4 - Machine Vision from 9 AM to 10 AM
5. Class 5 - Machine Learning from 9:30 AM to 10:30 AM

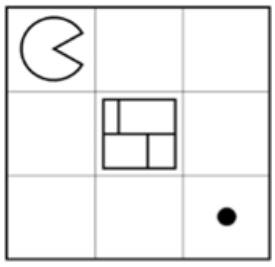
The professors are:

1. Professor A who can teach classes 3 and 4.
2. Professor B who can teach classes 2, 3, 4, and 5.
3. Professor C who can teach all classes.

a) Formulate this problem as a constraint satisfaction problem in such a way that there is a variable for each class, and also state their domains and constraints.

b) Draw the constraint graph for this constraint satisfaction problem.
c) Your CSP problem should have a tree-like structure. Briefly explain why we prefer to solve CSP problems with a tree structure.
d) Investigate what method exists to convert this type of problem with a semi-tree structure to a tree.

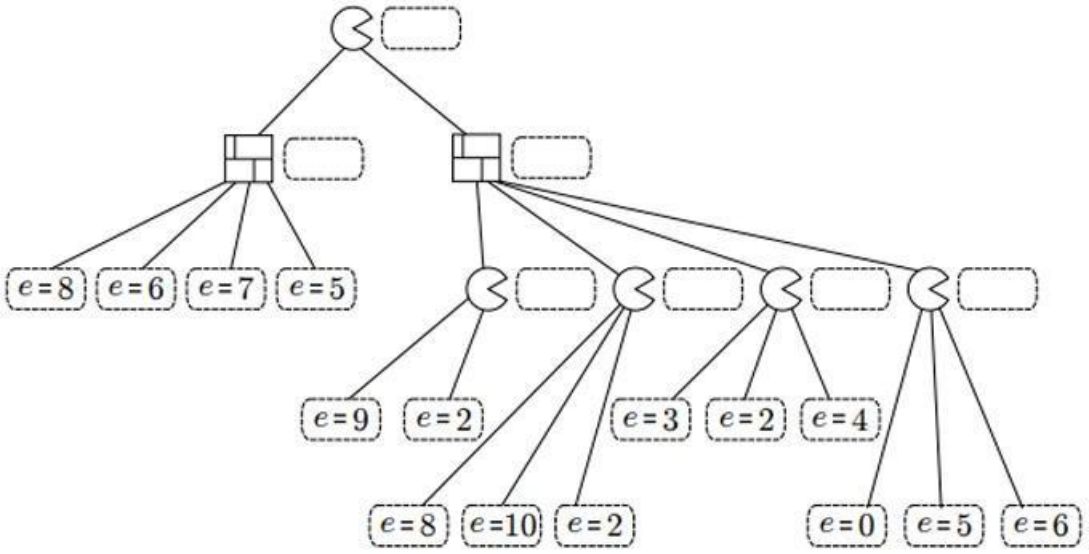
Q2: Pac-Man
Based on the following figure, in the Pac-Man game, the Pac-Man agent must play against the wall agent, and there is also a dot in one of the table's cells, which is the Pac-Man agent's target cell. This Pac-Man agent can move in one of the 4 main directions (up, down, left, and right) and occupy one of the unoccupied cells. On the other hand, when it's the wall agent's turn, this agent can also move towards one of the 4 main directions and occupy unoccupied cells. Furthermore, the wall cannot enter cells that contain a dot, and standing still is not possible for any agent when it's their turn. Pac-Man's score is equal to the number of dots he can reach, and each time Pac-Man reaches a cell that has a dot, that dot disappears from that cell, and appears in another cell that is not occupied, and the game continues. The goal of Pac-Man in this game is to reach the cells with dots to increase his score.



Assume the first move in the above figure's configuration starts.
According to the explanations provided, answer the following questions.

a) Create a game tree for a situation where each player is only allowed to make one move. (Only draw the allowed moves). Considering the tree with a limited depth that you have drawn, what is the game value (Pac-Man's score)? (The game starts with Pac-Man's move.)

b) Another game of the same type is played on a more complex board (game page). A part of the game tree is drawn in the figure below, and the leaf nodes are scored with an unknown evaluation function. Using the minimax algorithm, place appropriate values in the empty rectangles in the figure. Also, identify the leaves that are not examined by the alpha-beta pruning algorithm.



Q3: Cryptarithmetic Puzzle

Consider the following Cryptarithmetic Puzzle problem. Perform the backtracking algorithm with the MVR and LCV heuristics for it, and at each step, describe the variable you choose for assignment, the domain values of other variables after its assignment, and whether backtracking is necessary or not. (If the tree in question has large dimensions, drawing and examining a small part of it is sufficient.)

$$\begin{array}{r} SEND \\ +MORE \\ \hline MONEY \end{array}$$

Q4: consistency

Answer the following questions about consistency.

a) Can k-consistency be inferred from k+1-consistency?
b) In the problem below, examine arc-consistency and rewrite the domains of the variables using AC-3.

$A \in \{1,2,3,4,5,6\}$, $B \in \{1,4\}$

$C \in \{5,6,7\}$, $D \in \{6,7,8,9\}$

$A \leq B$ $A + C \leq 8$

$C = D$

Q5: minimax tree

In the game tree below, the MAX player makes the first move. In the leaves of the tree, the final score of the game for MAX is specified. Answer the following questions:

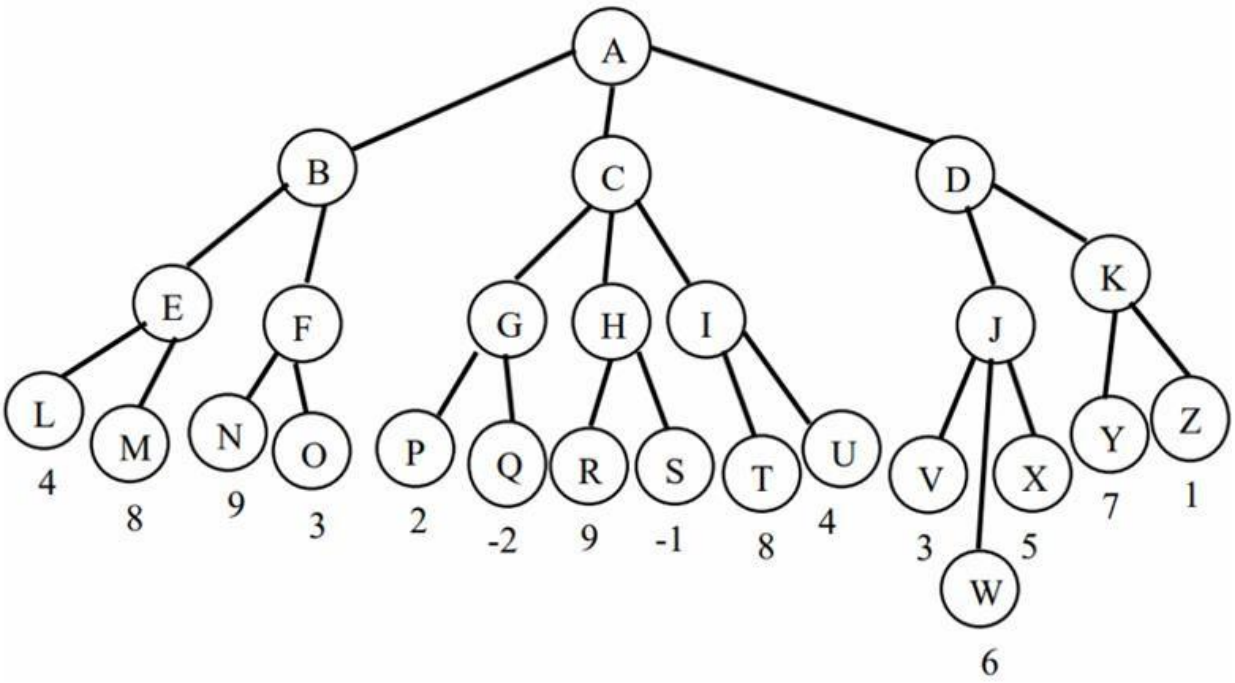
a) Specify the minimax value for the other nodes.

b) What is the first move chosen by MAX?

c) If the alpha-beta pruning algorithm is used, which nodes are pruned? (Assume the children of a node are visited from left to right.)

d) In general (not just this tree), if when traversing a game tree, we visit the child nodes from right to left instead of left to right, does any change occur in the minimax value calculated at the root node? Is it possible for there to be a change in the number of nodes pruned by the alpha-beta algorithm?

e) Redo the alpha-beta pruning operation on the tree below. This time, assume that the child nodes are visited from right to left. Which nodes are pruned?



Programming Questions

Q1-Title: *Conscious Drone Navigation in Adversarial Terrain*

Scenario

A commercial airplane has crashed somewhere within a fully randomized hazardous terrain of size 15×15. Your task is to build an autonomous rescue drone that can intelligently search for the crash site using informed search algorithms in a partially observable, stochastic, and adversarial environment—under severe time and resource constraints.

The drone must locate the airplane within 30 steps or fewer, while managing fuel, lives, and limited visibility. The terrain includes dynamically moving hazards and fuel stations, and is randomized both at start and periodically during execution. A single rescue goal (G) randomly placed and guaranteed reachable

The drone must start at position (0, 0) and find the goal in 30 steps or fewer while managing fuel, lives, and visibility.

Game Environment Summary

Feature	Description
Grid Size	15 × 15 grid (225 cells total)
Mountains (M)	Randomly placed in 20% of cells; drone loses 1 life and cannot enter the cell(no teleportation).
No-Fly Zones (X)	Hidden and placed in 20% of cells; invisible until entered; entering causes instant mission failure.
Energy Stations (E)	15 fuel stations randomly placed on the map; drone must refuel every 10 moves or lose 1 life.
Storms (~)	Appear randomly every 10 steps; invisible until collision. Collision causes: – 1 life lost – scan blocked – next move randomized
Rescue Goal (G)	Exactly one airplane crash site, randomly placed and always reachable; becomes visible only when within perception radius.

Terrain Randomization	Every 10 steps: – All terrain elements (mountains, storms, fuel stations) reshuffle – 2 new No-Fly Zones added – 1–3 new Storms generated

Drone Properties

Attribute	Value / Behavior
Movement	1 cell per turn in any of the 8 directions (up/down/left/right and diagonals)
Fuel	Starts with 10 units; must refuel every 10 moves or lose 1 life
Lives	Starts with 3 lives; lose 1 when hitting a mountain, storm, or running out of fuel
Perception	Radius = 2; limited field of vision (fog-of-war environment)
Scan	3 tiles forward; blocked by storms
Replanning	Required after any environmental reshuffle or upon hitting a hazard
Max Steps	30 steps is the hard limit to complete the mission
Collision (Mountain)	Drone loses 1 life, cannot enter the cell, remains in place (no teleportation)

Dynamic Hazards

- Storm Zones (~):
    - Appear dynamically every 10 steps

    - Block visibility and scan
    - Increase movement cost (randomized: 3–9)
- No-Fly Zones (X):
    - Invisible until adjacent
    - Entering results in instant mission failure
- Every 10 Steps:
    - +2 new No-Fly zones
    - +1–3 new storms
    - All terrain (energy, goal, mountains) reshuffles

Task Breakdown

Task 1: PEAS Model & Agent Architecture

- Define the PEAS components (Performance, Environment, Actuators, Sensors) for the drone agent.
- Design the agent using a hybrid architecture combining Goal-based and Utility-based behavior.
- Describe the Perception → Reasoning → Action loop, including how the drone interprets new data, makes decisions, and adapts after terrain changes.

Task 2: State-Space Modeling
Define the full state of the drone at each point using:
(x, y, fuel, lives, path_cost, known_no_fly_zones, known_storm_map, visible_map, time_step)
Your model must include:

- A Successor Function: defines legal actions and resulting states
- A Transition Model: describes how state changes with each action
- A Cost Function: g(n) should reflect realistic energy/life costs
- A Goal Test: returns true when the drone reaches the rescue site

Task 3: Heuristic Design & Evaluation

You may not use predefined heuristics.

You must design five (5) unique heuristic functions based on:

- Estimated distance to goal
- Remaining fuel
- Remaining lives
- Presence of obstacles and hazards (e.g., storms, no-fly zones, mountains)
- Step limit (max 30 steps)

For each heuristic:

- Clearly explain its mathematical formula
- Justify its logic and intended behavior
- Analyze its:
    - Admissibility: never overestimates
    - Consistency: $h(n) \leq cost(n \rightarrow n') + h(n')$
- Run A* using each heuristic and compare:
    - Final path cost
    - Number of nodes expanded
    - Execution time

➢ Creativity and realism in heuristic design will be rewarded.

Task 4: Algorithm Implementation

Implement and evaluate the following search algorithms:

Algorithm	Requirements
UCS	Baseline for cost-optimality
Greedy Best-First	Use two of your own heuristics
A* Tree Search	No closed set; may revisit states
A* Graph Search	Must use closed set; must handle dynamic terrain updates and replanning

All implementations must:

- Handle fully randomized environments

- Support real-time replanning
- Avoid infinite loops or redundant paths

Task 5: Dynamic Goal Switching

- The goal (G) is unknown at the start and only becomes visible when within the drone's perception radius.
- Once discovered, the agent must replan optimally toward the goal.
- Only one fixed goal exists; no dynamic goal switching is required.

Task 6: Logging & Animation

Logging must include:

- Drone’s position and action per step
- Fuel and life status
- Replanning triggers
- Total number of nodes expanded

Live animation must:

- Use matplotlib + IPython.display.clear_output()
- Display:
    - Drone movements
    - Perception field (fog-of-war)
    - Terrain changes (storms, fuel, mountains)

Final screen must show:

MISSION ACCOMPLISHED

Lives: X | Fuel: Y | Nodes Expanded: Z

Heuristic Visualization:

- For each algorithm and heuristic, show:
    - h(n) heatmap
    - g(n) path cost so far
    - f(n) estimated total cost

Submissions without live animation and heuristic plotting will lose significant marks.

- *Assignments without proper live animation and heuristic plotting will lose marks.*

Deliverable

Submit a formal report with:

- A summary of all five heuristics
- Analysis of their admissibility and consistency
- Performance comparison for each algorithm + heuristic combo:
    - Path cost
    - Time
    - Node expansion
- Charts and heatmaps
- Final reflection: Which strategy performed best and why?

Bonus Tasks (+10 pts)

Bonus Feature	Points
Learned heuristic from relaxed environment	+2
Real-time adaptive fog-of-war rendering	+2
Storm-aware f(n) heatmap	+2
Firework animation upon goal reach	+1
Probabilistic tie-breaking	+1
Fuel-aware smart rerouting	+2

Allowed Python Libraries

You may use only the following Python libraries:

- random, math, time, copy
- numpy, matplotlib, IPython.display

Q2-Title: Strategic Diplomacy AI

Dynamic Constraint Satisfaction and Adversarial Multi-Agent Negotiation

Scenario

You are tasked with designing a multi-agent AI system to manage a 3-day diplomatic summit among six rival factions, where global resources—Energy, Water, and Technology—must be strategically negotiated and redistributed. Your AI must integrate:

1. A robust Constraint Satisfaction Problem (CSP) system for adaptive scheduling
2. A dynamic Adversarial Search engine for zero-sum, partially observable negotiation games

This simulation blends resource allocation, scheduling under failure, strategic reasoning, bluffing, and game tree evaluation into a single end-to-end AI system.

Part A – Adaptive Diplomatic Scheduling (CSP)

Objectives

Design a CSP system to assign:

- 3 negotiation sessions per day
- 3 days, 3 rooms per day
- Each session involves 2 unique factions

Constraints :

- No faction meets another more than once
- No faction uses the same room in two consecutive sessions
- Factions must have at least one slot break between sessions on a single day
- Rival factions can only meet in neutral rooms

Dynamic Element:

At any point during the summit, one room may be randomly destroyed (unavailable). Your system must:

- Detect and resolve the failure without recomputing the entire schedule

- Implement rescheduling through rollback or constraint repair techniques

Implementation

- CSP modeling (variables, domains, constraints)
- Backtracking with:
    - MRV (Minimum Remaining Values)
    - Degree heuristic
    - Forward Checking
    - Arc Consistency (AC-3)
- Rescheduling module for dynamic constraint changes

Output

- Final 3D schedule: Day × TimeSlot × Room → (Faction1, Faction2)
- CSP trace and domain size evolution (optional but recommended)

Part B – Strategic Resource Negotiation (Adversarial Search)

Overview

Each session becomes a zero-sum negotiation game between two factions. The outcome depends on strategies, fatigue, priorities, and history.

Game Setup:

- Available actions: {Offer, Threaten, Bluff, Concede, Delay}
- Utilities are affected by:
    - Each faction's resource urgency
    - Fatigue from back-to-back sessions (from Part A)
    - Room bias (neutral vs. owned)
    - Previous encounters (trust or rivalry)

Advanced Extensions:

- Multi-Round Memory:
  Factions adjust strategies based on negotiation history
- Partial Information Gameplay:
  Opponent does not know true resource priorities
- Learning-Based Evaluation Function *(Required)*:
  Tune your evaluation logic using logs from self-play sessions
  (No ML libraries allowed; manual/statistical tuning only)
- *(Optional)* Coalition Rounds:
  Three-party negotiations with shared utilities (non-zero-sum extension)

Implementation

- Game engine using:
  - Minimax with Alpha-Beta pruning
  - Depth-limited tree search
  - Three distinct custom evaluation functions
- Analysis of:
  - Admissibility and Consistency
  - Simulation outcomes using each function

Output

- Utility results per match
- Nodes expanded and time per strategy
- Game tree visuals for at least 3 sessions
- Plots of evaluation performance over time

Deliverables

- PDF report containing:
  - CSP formulation and dynamic handling
  - Game tree screenshots and evaluation design
  - Strategy outcomes, performance metrics
  - Analysis: impact of fatigue, history, and partial info

Bonus Features (Up to +10 Points)

Bonus Feature	Points
Learned evaluation function (via self-play)	+2
Room failure with constraint repair	+2
History-based strategy adaptation	+2
Partial information + bluff detection	+2
Coalition-based non-zero-sum gameplay	+2

Allowed Python Libraries

You are allowed to use only the following:

- random, math, copy, time
- numpy, matplotlib, IPython.display
- Do not use external AI/game libraries.All algorithms (CSP, Minimax, evaluation) must be implemented from scratch.

Important Notes

- Two zip files, one containing the answers to the theory questions in PDF format and the other containing the answers to the programming questions in Jupiter Notebook format and a report in PDF format. Be sure to upload both to Quera and your score will be calculated based on the files uploaded to Quera.
- Be sure to upload the assignment by the deadline specified in Quera. The maximum deadline for late upload is 1 day and there is a penalty and you will only be awarded 70% of the grade and no more grade will be awarded after that.
- Academic Integrity Any form of academic dishonesty will result in a zero grade for this assignment.
- You are allowed to use tools like GPT to assist you in writing code or understanding the concepts, but it is crucial that you fully understand the code you write and how it works. You may be required to explain your implementation during an assessment.

- Code Quality: Your code should be well-documented and follow good programming practices.
- You need to implement the algorithms from scratch. Using the built-in functions and algorithms is not allowed.
- Provide a report in PDF format and explain your code and results.
- The name of the uploading file should be "Firstname_Lastname_StdNumber.zip" of all group members.
- Collaboration Policy: While discussions with peers are allowed, your submitted work must be original.
- Late Submissions: Late submissions may incur penalties as per course policy.
- If you have any questions about anything, feel free to ask in the course's group chat.

Good Luck!